From-Scratch Wide ResNets Under a 5M Parameter Cap: A Study on the CIFAR-10 Variant Challenge

Team: Bonnie Chen, Xinquan Wang, Qinghao Shen . **NYU DL 26 Fall, Project 1 Repository:** https://github.com/NYUWXQ/DL26fall.git **Competition:** cifar-10-res-net-architecture-challenge-dl

> Draft status: numbers in this draft come exclusively from `python src/models.py` (run 2026-10-31, torch 2.15.0 / MPS), `runs/*/history.json`, and figures verified by the team lead. Entries marked **[PENDING]** await the in-progress training run and will be finalized before submission. No number in this document may be cited unless it appears in one of those sources.

1. Introduction

Building image classifiers "from scratch" forces a different kind of engineering than fine-tuning: every trick normally imported with a pretrained backbone must be re-derived, and the parameter budget of **5,000,000** constrains architecture choices before a single epoch is spent. This report documents our pipeline for the CIFAR-10 variant challenge: a pre-activation Wide ResNet, a training regime of strong augmentation and mixed-label losses (RandAugment, MixUp/CutMix, label smoothing), an inference stage with translation+flip TTA and checkpoint ensembling, and an analysis of why the public leaderboard saturates near 0.909 regardless of local accuracy.

Two facts frame everything that follows. First, a teammate's earlier pipeline (`proj1/`, a WRN-16-84 trained with `torchvision.datasets.CIFAR10`) reached **97.48%** on a held-out 5,000-image set while scoring only **0.906** on the public leaderboard - a **6.9-point** gap. Second, the current leaderboard leader sits at **0.909**, i.e. at most 0.3 above that baseline. Any architecture-level paper improvement therefore matters far less than (a) understanding the gap and (b) cheap, composable accuracy gains: ensembling and TTA. We structure the report accordingly.

2. Data and Problem Setup

The competition ships **50,000** labeled training images and **2,000** test images as 32x32 RGB PNG files with CSV manifests (`train_labels.csv`, `test.csv`). Because the competition images are packaged as PNG rather than through `torchvision.datasets`, the first question is whether they are byte-identical in distribution to standard CIFAR-10 (Section 5 evaluates this).

Our data path (`src/data.py`):

1. **One-time materialization.** PNGs are decoded to a `(N,3,32,32)` uint8 numpy cache (`.cache/train_x.npy`, etc.), so repeated epochs pay no decode cost - at ~35 s/epoch on the Apple-MPS dev setup, this is the difference between a workable and an unworkable iteration loop.
2. **Deterministic validation split.** With a fixed seed, we hold out **2,500** of the 50,000 images as validation, leaving 47,500 for training. Every reported validation number below refers to this split.
3. **Normalization** by the standard CIFAR-10 channel statistics (mean `(0.4914, 0.4822, 0.4465)`, std `(0.2470, 0.2435, 0.2616)`), a choice we keep because the shipped images are CIFAR-10-derived.

Augmentation differs by regime (`src/data.py`, `--aug`):

- `basic`: random crop with 4-pixel padding + horizontal flip.
- `strong`: the above, plus **RandAugment** (num_ops=2, magnitude=9) and **RandomErasing** (p=0.25, scale 0.02-0.2). This is always-on in our training runs.

At the batch level (`src/train.py`, `--mix both`) we apply **MixUp or CutMix with probability 0.5 each** (per-step coin flip, then uniform choice, alpha=1.0), and use label smoothing 0.1.

3. Architecture and the Parameter Budget

3.1 Model family

`src/models.py` implements a configurable **pre-activation ResNet** (`PreActBlock` + `PreActResNet`) for 32x32 inputs: each block computes `y = x + F(BN-ReLU-Conv-BN-ReLU-Conv(x))`, with a 1x1 projection shortcut only when the feature-map shape changes, and a global-average-pool + linear head. The number of blocks is `(depth-4)/6` across three stages with widths `[16w, 32w, 64w]`, following the Wide-ResNet convention.

3.2 The 5M cap enforces itself

We treat the budget as a program invariant, not a design note: `check_budget()` raises before training starts if a model exceeds 5,000,000 trainable parameters. The production table, printed by `python src/models.py` on this machine:

model name	params (exact)	within budget
preact28x2	1,472,170	yes

model name	params (exact)	within budget
preact16x4	2,776,906	yes
preact22x4	**4,326,986**	yes (production choice)
preact16x6	6,242,026	no rejected at startup
preact16x8	11,091,594	no rejected at startup

The interesting trade-off the table exposes: depthxwidth inside this budget is nearly a zero-sum game. `preact22x4` spends the cap; `preact16x6` and `preact16x8` overshoot it and are excluded by the guard rather than by judgment, which is exactly the point - an accidental 5M+ submission cannot happen through our entry point.

Our reasoning for `preact22x4`: among candidates in-family, it is the deepest x widest configuration that still fits the cap, following the wide-residual-network principle that width buys more accuracy per parameter than depth on small images. The teammate's independent WRN-16-84 line (97.48% holdout) points the same way: their strongest result came from a wide shallow stage configuration, not from extra depth.

4. Training Recipe

`src/train.py` trains with an explicit optimizer stack (no external training framework), which we deliberately kept in-code so the paper can cite exact settings:

- **Optimizer:** SGD momentum **0.9**, Nesterov, weight decay **5e-4**.
- **Schedule:** linear warmup (5 epochs) then cosine decay to 0; production uses **LR=0.25, batch size 512, 300 epochs** (scaled to the machine's capability; HPC runs use the same script via Slurm `slurm/train.sbatch`).
- **Loss:** cross-entropy with label smoothing 0.1 and the MixUp/CutMix lam blend of Section 2.
- **EMA:** exponential moving average of the weights with **decay 0.9995**, evaluated alongside the raw model at every fifth epoch; `src/predict.py` prefers EMA weights when present ("the usual winners").
- **Reproducibility:** seeded split, seeded torch/numpy state, full args saved in every checkpoint dict; training resumes from a checkpoint by re-reading its saved args (architecture, augmentation, mix regime) before rebuilding the model and restoring optimizer state, step counter, and history.

4.1 Observed trajectory (in-progress run)

The production run (`runs/wrn22x4_s0`, preact22x4, seed 0) is in progress. Verified so far (`runs/wrn22x4_s0/history.json`):

epoch	train loss	val acc (raw)	val acc (EMA)
5	1.4378	0.634	0.0892

The raw model at 63.4% after 5 epochs is on the expected trajectory for this recipe; the EMA shadow is only 8.9% at that point because it lags heavily during warmup (it is a weighted average of recent iterates while the model is moving fastest). We report both to be transparent that early EMA numbers are not diagnostic. **[PENDING]** Final validation accuracy, convergence curve, and loss history will be filled from `runs/wrn22x4_s0/history.json` once training completes.

5. Results and Analysis

5.1 The gap, three hypotheses, and what the evidence says

source	split	accuracy	leaderboard
teammates' original pipeline (`proj1/`, WRN-16-84, from-scratch)	5,000-image holdout	**97.48%**	**0.906**
current public leaderboard	-	-	**0.909 (top)**

The 6.9-point val->LB drop is the single most consequential number in this project. Three candidate explanations were on the table, each with a different, actionable fix:

- **(H1) Leaderboard-label noise.** If the LB labels are noisy but the val split is clean, the leaderboard is a downward-biased estimator of true accuracy, and the fix is to maximize true accuracy and compose it with ensembling.
- **(H2) Photometric/encoding domain shift.** If the shipped test PNGs differ from standard CIFAR-10 in brightness, contrast, or encoding, local accuracy overstates generalization, and the fix includes distribution matching at training or inference time.
- **(H3) A re-collected, harder test set.** The test images could be newly gathered data of the same 10 classes but a different collection distribution - the regime studied in the literature as benchmark censorship / re-collected test sets, where reported drops range from roughly 3 to 11 points.

Verdict (measured, reproducible). The team lead's comparisons settle the hypothesis split:

1. **Training data is stock CIFAR-10.** A 4,000-image pixel-exact comparison of competition `train_images/` against the official CIFAR-10 training set (Kaggle mirror, CDN-pulled) matched **4,000/4,000** by pixel-level digest.
2. **The test set is neither standard CIFAR-10 test nor CIFAR-10.1.** Against the official CIFAR-10 test set: **0/2,000** matches. Against CIFAR-10.1 (Recht et al.'s 2,000-image re-collected set): **0/2,000**. Across the full dihedral group (8 transforms), only horizontal flip hits 18/2,000 - negligible. The test pool is therefore **another 2,000-image collection of the same 10 classes**, not a transform of anything we hold.
3. **The photometric deltas are real but tiny in effect.** Applying the measured test-set statistics to the 2,500 labeled validation images (epoch-40 checkpoint):

transform applied to val	val acc	delta
baseline	0.8548	-
x.1.05 + 8 (brightness +4.4/255, contrast +5%)	0.8568	+0.0020
x.1.04 (brightness only)	0.8540	-0.0008
gamma 0.92	0.8552	+0.0004
quantize step 4	0.8560	+0.0012
gaussian blur sigma~1 *(negative control)*	0.2784	-0.5764

The four photometric transforms reach at most **+/-0.2 points** - they cannot explain a 6.9-point gap. The negative control shows the experiment would have detected a real shift. Gradient-energy statistics (test 15.34 vs train 14.38) confirm the shipped test images are not blurred.

What remains. H2 is **rejected** (above). H1 cannot be verified - we hold no ground-truth labels for the leaderboard test set - so **the report makes no claim about label noise**, and any earlier derivation-only estimate along those lines is withdrawn from this document. What the evidence supports is the plain reading: **the leaderboard test set is a re-collected 2,000-image set that is simply harder**, and 0.906 is a normal score for it; literature on re-collected test sets reports typical drops of 3-11 points, placing our 6.9-point drop squarely in the expected band. The top score 0.909 then reads as all teams hitting the same ceiling - which redirects effort from domain matching to the only levers that work on a harder test pool: **OOD generalization via ensembling and TTA**, plus making the single model as strong as possible within 5M parameters.

> A controlled quantification of the IID->re-collected gap (our checkpoints on the official CIFAR-10 test set vs CIFAR-10.1, diagnosis-only) is in progress in `analysis/lb_noise.md`; it will replace this paragraph's expectations with measured deltas.

5.2 Inference, TTA, ensembling, and the domain-shift evidence

`src/predict.py` supports four inference modes (`plain`, `flip`, `translate`, `translate_flip`) and an arbitrary list of checkpoints whose softmax outputs are averaged. The production configuration decodes as:

- Base view: the image as shipped.
- Translate views: reflect-pad-2 then crops at offsets (0,0), (0,+4), (+4,0), (+4,+4) - 4 shifted crops.
- Each view is also horizontally flipped: **10 views total**.
- Softmax probabilities averaged across views *and* checkpoints; argmax is the final label.
- Output CSV preserves `sample_submission.csv` row order exactly, with a per-submission metadata sidecar (checkpoint list, TTA mode, label histogram, SHA-256 of the CSV) for audit.
- **Mechanism caveat (explicit).** Recomputing BatchNorm statistics on the test set is *transductive* - it consumes unlabeled test inputs at inference time. If BN recalibration were ever adopted, the report would state that explicitly and never present it alongside purely supervised results. As of this draft it is **not** part of the submission path (the controlled experiment in Section 5.1 independently showed no usable domain effect for it on this data).

Interim evidence, superseded by Section 5.1's verdict (kept for the record). The confidence-profile comparison that first suggested a shift - val n=2500: mean confidence 0.6511, entropy 1.217, balanced predicted-class mass; test n=2000: mean confidence 0.6031 (-4.8 points), entropy 1.310, visibly skewed predicted-class mass - plus the pixel statistics (+4.4/255 brightness, ~5% higher contrast) motivated the controlled experiment in Section 5.1. That experiment then showed these deltas cost at most +/-0.2 points, and the pixel-id comparisons showed the test pool is re-collected rather than transformed. We keep this paragraph because the diagnostic chain (confidence gap -> pixel statistics -> controlled transform experiment -> pixel-id comparison) is how the hypothesis was actually tested and killed.

[PENDING] A dedicated inference-configuration comparison (plain / flip / translate / translate_flip, with and without EMA-weight checkpoints, run by `src/refine.py`) will report the marginal value of each stage. We expect the ordering to be: EMA > raw, translate_flip > flip > plain, and ensemble N>1 > single checkpoint, but publish only what has been

measured. One measured intermediate (epoch 31 checkpoint): horizontal-flip TTA adds **+1.52 points** (0.8576 -> 0.8728, McNemar p=0.0004); we report it as supporting the TTA direction, not as a final number.

5.3 What the constraints let us aim for

With H2 rejected, the earlier noise-implied arithmetic target is retired alongside it: no defensible accuracy threshold can be derived while the gap's cause stays unverified. What stands is what the evidence supports:

- The leaderboard pool is re-collected, harder data of the same 10 classes; per the re-collected-test-set literature (3-11 points), our 6.9-point val->LB drop is a normal data-difficulty effect, not a fixable bias.
- The only levers that help on a harder OOD test pool are **true accuracy** (single model as strong as the 5M cap allows) **composed with ensembling and TTA**, where ensembling tends to pay off more on OOD than on IID.
- A real LB submission from our final checkpoint will measure the val->LB mapping directly instead of modeling it.

Our claim to improvement is therefore *methodological* end to end: (1) from-scratch training under a hard param cap, (2) a reproduced, exactly-documented recipe, (3) 10-view TTA + multi-checkpoint ensembling as the accuracy multiplier, and (4) an audit trail (`scripts/check_deliverables.py`, per-submission SHA-256 metadata) that protects the 5M/no-pretrain/no-external-data constraints programmatically rather than by promise. Because the gap is data difficulty, not pipeline error, we do not promise an LB number - we promise a measured one after the first submission.

5.4 Compliance audit

The assignment forbids pretrained weights, external datasets, and >5M parameters. We enforce this with both code and grep: `check_budget()` guards every model construction, submission-time loaders re-derive the parameter count from the loaded state dict, and `scripts/check_deliverables.py` re-runs a clean-checkout pass that greps `src/` for `pretrained=|from_pretrained|torch.hub` and validates CSV row alignment/shape (2,000 predictions, ids matching `sample_submission.csv` order).

6. HPC Deployment

The same pipeline runs on NYU Torch HPC via `slurm/train.sbatch` + `notebooks/hpc_train.ipynb` (Apptainer/Singularity with the CUDA 12.1.1/CUDNN 8.9.0 image, conda overlay mounted `:ro`, `--account=` per `my_slurm_accounts`, QoS-based partition selection - no hardcoded partition, preemption-borrowing enabled via `--comment="preemption=yes;requeue=true"`). Compute-node internet restrictions are documented in `docs/HPC.md`; the code+data bundle is rsynced from the login node into `/scratch/$USER/dl26fall/`. Details are external to the paper's scientifically meaningful claims; see that notebook for the reproducible step-by-step.

7. Discussion and Conclusions

What the numbers say (verified only). The architecture-budget table (Section 3.2), the training trajectory (Section 4.1), the pixel-id verdict and controlled-transform table (Section 5.1), and the TTA intermediate (Section 5.2) are all reproducible from commands and files this report names. The final accuracy of the production run is not yet citable and stays **[PENDING]** rather than estimated.

What we expect to learn next. (1) Final single-model accuracy on the 2,500-image split, (2) the TTA/ensemble decomposition from `src/refine.py` on `final.pt` (raw/EMA reported separately), (3) the measured IID-vs-re-collected gap from the `analysis/lb_noise.md` diagnosis (checkpoints on the official CIFAR-10 test set vs CIFAR-10.1, diagnosis-only), and (4) the val->LB mapping measured directly by our first real submission - replacing every modeled expectation with a data point.

Ensembling plan (design, results pending). The training pipeline is adding two decorrelated members via the overnight queue: `preact16x4` seed 1 and `preact28x2` seed 2 (both inside the 5M cap at 2.78M / 1.47M params), with the validation split fixed to seed 0 across members so per-member val accuracy stays comparable. The rationale is error decorrelation rather than squeezing a single model: on a re-collected OOD pool, ensembles typically gain more than they do on IID. We report per-member val numbers and the ensemble's val gain once each member finishes - no modeled extrapolations.

One dry-run data point (submission pipeline, not a result). Before final weights exist, the full submission path was exercised on the epoch-40 checkpoint: `submissions/dryrun_epoch40.csv` came out with 2,000 rows, exact `sample_submission.csv` id order, `id,label` columns, in 21.2 s with flip TTA. The skewed class histogram in that dry run reflects mid-training weakness, not a final distribution; it validates plumbing only.

Takeaways for the reader. Under a hard parameter cap, the biggest wins come from outside the architecture: strong augmentation (RandAugment + RandomErasing), MixUp/CutMix blending, EMA weights, and inference-time TTA/ensembling. And when local scores drop on a leaderboard, the most valuable engineering artifact is not a better model - it is the diagnostic chain that locates *why*: here, pixel-id comparisons and a controlled transform experiment replaced a plausible (and wrong) attribution with a plain one. The gap is data difficulty; the response is generalization, not domain matching.

References

1. Zagoruyko, S., & Komodakis, N. (2016). *Wide Residual Networks*. BMVC.
2. He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Identity Mappings in Deep Residual Networks*. ECCV.
3. Cubuk, E. D., et al. (2020). *RandAugment: Practical automated data augmentation with a reduced search space*. CVPR.
4. Zhong, Z., et al. (2020). *Random Erasing Data Augmentation*. AAAI.
5. Zhang, H., et al. (2018). *mixup: Beyond Empirical Risk Minimization*. ICLR.
6. Yun, S., et al. (2019). *CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features*. ICCV.
7. Szegedy, C., et al. (2016). *Rethinking the Inception Architecture* (label smoothing). CVPR.
8. Hoffer, E., et al. (2017). *Train longer, generalize better: closing the generalization gap in large batch training*. arXiv:1707.09895.
9. Recht, B., Roelofs, R., Schmidt, L., & Shankar, V. (2019). *Do CIFAR-10 Classifiers Generalize to CIFAR-10.1?* ICCV.
10. Recht, B., et al. (2018). *Do ImageNet Classifiers Generalize to ImageNet?* ICML.
11. Kaggle competition page: cifar-10-res-net-architecture-challenge-dl.

(No provisional citations: every reference above is a verifiable published paper or the competition page itself. References 9-10 back the re-collected-test-set literature claim in Section 5.1; the 3-11 point band is theirs. If Section 5.3's diagnosis turns it into a measured claim, we cite the measured run in the text and keep 9-10 as framing.)